Lab Report Submission

Course Code: CSE 314

Course Name: Compiler Design Lab

Experiment No: 02

Experiment Name: Introduction to Flex and Bison — Lexical Analyzer, Arithmetic Expression Parser, Variable Assignment Parser, Boolean Expression Parser, If-Else Statement Parser, and List of Numbers Parser

Submitted To

Name: Rabeya Khatun

Designation: Lecturer

Department: CSE

Daffodil International University

Submitted By

Name: MD. Emon Hossain

ID: 232-15-818

Section: 66_L2

Semester: Spring 2026

Department: CSE

Daffodil International University

Index

Lab No.	Experiment Name	Page
05	Introduction to Flex — Lexical Analyzer	2–14
06	Flex & Bison — Arithmetic Expression Parser	15–20
07	Flex & Bison — Variable Assignment Parser with Symbol Table	21–26
08	Flex & Bison — Boolean Expression Parser	27–32
09	Flex & Bison — If-Else Statement Parser	33–38
10	Flex & Bison — List of Numbers Parser	39–43

Experiment No. 05: Introduction to Flex — Lexical Analyzer

1. Objective

- To understand the concept and structure of Flex (Fast Lexical Analyzer).
- To write and run basic Flex programs for token recognition.
- To recognize numbers, words, and whitespace using Flex rules.
- To count vowels and consonants using Flex.
- To count arithmetic operators (+, -, *, /) using Flex.
- To distinguish keywords from identifiers using Flex.
- To design a complete lexical analyzer using Flex.

2. Software Requirement

- Flex (Fast Lexical Analyzer) — version 2.6.4
- GCC C Compiler (MinGW / GCC on Linux)
- IDE / Text Editor: VS Code / CodeBlocks
- Operating System: Linux / Windows (WSL or WinFlexBison)

3. Theory

Flex (Fast Lexical Analyzer) is an open-source tool for generating scanners — programs that break input text into tokens (words, numbers, symbols). It reads a .l file (containing scanner rules) and generates C code (lex.yy.c). That C code is then compiled to produce a working lexer. Flex is widely used in the front end of compilers for lexical analysis, which is the first phase of compilation.

3.1 Basic Structure of a Flex File

A Flex file has three sections separated by %%:

- Section 1 (%{ ... %}) — C declarations and header includes
- **Section 2 (%% ... %%) — Pattern rules and actions (the scanner rules)**
- **Section 3 (%% ... end) — C function definitions such as main()**

3.2 Key Built-in Variables and Functions

Component	Meaning	Purpose
yytext	Current matched token (char*)	Access the matched text (e.g., print or convert it)
yywrap()	End-of-input handler	Return 1 to signal end of scanning; return 0 for more files
yylex()	Main scanner function	Start lexical analysis — reads input and matches tokens

3.3 How to Run a Flex Program

1. Step 1: Write the Flex source file (e.g., example.l)
2. Step 2: Run Flex: flex example.l → generates lex.yy.c
3. Step 3: Compile: gcc lex.yy.c -o example
4. Step 4: Execute: ./example

3.4 Common Regular Expressions in Flex

Regular Expression	Description	Example Matches
[a-zA-Z_][a-zA-Z0-9_]*	Identifier (variable/function names)	myVar, _temp, x1
[0-9]+	Integer numbers	42, 123, 0
[0-9]+\.[0-9]+	Floating point numbers	3.14, 0.99
"if", "else", "while"	Keywords (matched as string literals)	if, else, while
[\t\n]+	Whitespace (space, tab, newline)	(usually ignored)
.	Any other single character (catch-all)	@, #, ^

4. Programs

Note: Programs are written as .l (Lex/Flex) source files. Each program demonstrates a specific lexical analysis concept using Flex pattern rules and C actions.

Program 1: Recognize Numbers, Words, and Whitespace Characters

Code (example.l):

```
%{
#include<stdio.h>
int count = 0;
%}
%%
[0-9]+            {printf("Numbers: %s\n", yytext);}
[0-9]+\.[0-9]+    {printf(" Float Numbers: %s\n", yytext);}
[A-Za-z]+         {printf("Words: %s\n", yytext);}
[ \t\n]           {printf("White space characters\n");}
.  ;
%%
int yywrap(){ return 1; }
int main()
{
   yylex();
   return 0;
}
```

Input & Output:

Input: hello 123 3.14

Output:

Words: hello
White space characters
Numbers: 123
White space characters
Float Numbers: 3.14

Program 2: Count Vowels and Consonants

Code Using if-else (vowel_consonant.l):

```
%{
#include <stdio.h>
int vowel_count = 0;
int consonant_count = 0;
%}
%%
[A-Za-z] {
  char c = yytext[0];
  if (c=='a'||c=='A'||c=='e'||c=='E'||
      c=='i'||c=='I'||c=='o'||c=='O'||
      c=='u'||c=='U') {
    vowel_count++;
  } else {
    consonant_count++;
  }
}
.\|\n ;
%%
int yywrap() { return 1; }
int main() {
  yylex();
  printf("Total vowels: %d\n",
         vowel_count);
  printf("Total consonants: %d\n",
         consonant_count);
  return 0;
}
```

Input & Output:

Input:
Hello World

Output:
Total vowels: 3
Total consonants: 7

Program 3: Count Arithmetic Operators (+, -, *, /)

Code (operators.l):

```
%{
#include <stdio.h>
int plus_count = 0;
int minus_count = 0;
int star_count = 0;
int slash_count = 0;
%}
%%
"+" { plus_count++; }
"-" { minus_count++; }
"*" { star_count++; }
"/" { slash_count++; }
.\|\n ; // Ignore other characters
%%
```

```
int yywrap() { return 1; }
int main() {
  yylex();
  printf("Total '+': %d\n", plus_count);
  printf("Total '-': %d\n", minus_count);
  printf("Total '*': %d\n", star_count);
  printf("Total '/': %d\n", slash_count);
  return 0;
}
```

Input & Output:

Input: a + b - c * d / e + f

Output:

```
Total '+': 2
Total '-': 1
Total '*': 1
Total '/': 1
```

Program 4: Identify Keywords vs Identifiers

Code (keywords.l):

```
%{
#include <stdio.h>
%}
%%
"int"   { printf("KEYWORD: %s\n", yytext); }
"if"    { printf("KEYWORD: %s\n", yytext); }
"while" { printf("KEYWORD: %s\n", yytext); }
[A-Za-z_][A-Za-z0-9_]*  { printf("IDENTIFIER: %s\n", yytext); }
[ \t\n]+ ;
. ;
%%
int yywrap() { return 1; }
int main() { yylex(); return 0; }
```

Input & Output:

Test Case–1:

Input: int x while count

Output:

```
KEYWORD: int
IDENTIFIER: x
KEYWORD: while
IDENTIFIER: count
```

Program 5: Design a Complete Lexical Analyzer

Code (lexer.l):

```
%{
```

```
#include <stdio.h>
#include <string.h>
char *keywords[] = {
  "int","float","if","else","while","for",
  "return","char","double","void","do",
  "break","continue","switch","case"
};
int isKeyword(char *word) {
  for(int i=0; i<15; i++)
    if(strcmp(word, keywords[i])==0) return 1;
  return 0;
}
%}
%%
[ \t\n]+                ;
"=="|"!="|"<="|">="    { printf("OPERATOR: %s\n", yytext); }
"+"|"-"|"*"|"/"|"="|"<"|">"  { printf("OPERATOR: %s\n", yytext); }
[0-9]+(\.[0-9]+)?        { printf("NUMBER: %s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]* {
  if(isKeyword(yytext))
    printf("KEYWORD: %s\n", yytext);
  else
    printf("IDENTIFIER: %s\n", yytext);
}
[(){}\[\];,]            { printf("SPECIAL CHARACTER: %s\n", yytext); }
.                       { printf("UNKNOWN: %s\n", yytext); }
%%
int main() { yylex(); return 0; }
int yywrap() { return 1; }
```

Input & Output:

Input: int a = 25 + b;

Output:

```
KEYWORD: int
IDENTIFIER: a
OPERATOR: =
NUMBER: 25
OPERATOR: +
IDENTIFIER: b
SPECIAL CHARACTER: ;
```

5. Observations

- Flex rules are matched from top to bottom — the first matching rule wins.
- The yytext variable holds the currently matched text, and yyleng holds its length.
- Longer matches take priority over shorter ones (longest match rule).
- The yywrap() function must return 1 to signal end of input.
- C code embedded inside %{ %} is copied directly to the generated lex.yy.c file.

6. Result

All objectives of this experiment were successfully achieved. The Flex programs compiled and executed correctly, producing expected outputs for each task: recognizing tokens, counting vowels/consonants, counting operators, distinguishing keywords from identifiers, and performing full lexical analysis.

7. Conclusion

This experiment introduced the Flex tool as a practical method of implementing lexical analysis — the first phase of compilation. Understanding Flex patterns and built-in variables such as yytext, yylex(), and yywrap() provides a solid foundation for building complete lexer-parser systems in compiler design.

Experiment No. 06: Flex & Bison — Arithmetic Expression Parser

1. Objective

- To understand the role of Bison as a parser generator in compiler design.
- To learn how Flex and Bison work together in a compiler pipeline.
- To parse and evaluate arithmetic expressions with +, −, *, /, and parentheses.
- To implement operator precedence and associativity in a grammar.

2. Software Requirement

- Flex 2.6.4 — Lexical Analyzer Generator
- Bison 3.x — Parser Generator (YACC-compatible)
- GCC C Compiler
- Operating System: Linux / Windows WSL

3. Theory

Bison is a parser generator that converts a Context-Free Grammar (CFG) into a C program that parses the grammar. It is used together with Flex: Flex handles tokenization (lexical analysis) and Bison handles syntax analysis (parsing). Bison generates a parse table from the grammar and implements an LALR(1) parser.

In this experiment, the arithmetic grammar is defined using three non-terminals to enforce correct operator precedence: expr handles + and −, term handles * and /, and factor handles numbers and parenthesized expressions.

4. Program Implementation

Flex File (calc.l) — Lexical Analyzer:

Code:

```
%{
#include "calc.tab.h"
%}
%%
[0-9]+      { yylval = atoi(yytext); return NUMBER; }
[ \t]       ;
[+\-*/\(\)] return yytext[0];
\n          return '\n';
.           return yytext[0];
%%
int yywrap() { return 1; }
```

Bison File (calc.y) — Parser:

Code:

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
```

```
void yyerror(const char *s);
%}
%token NUMBER
%%
input:
   input expr '\n' { printf("= %d\n", $2); }
   | /* empty */
   ;
expr:
   expr '+' term  { $$ = $1 + $3; }
   | expr '-' term  { $$ = $1 - $3; }
   | term
   ;
term:
   term '*' factor  { $$ = $1 * $3; }
   | term '/' factor  { $$ = $1 / $3; }
   | factor
   ;
factor:
   NUMBER
   | '(' expr ')'  { $$ = $2; }
   ;
%%
void yyerror(const char *s){ printf("Syntax Error: %s\n", s); }
int main() { printf("Enter expression:\n"); yyparse(); return 0; }
```

Compile & Run Commands:

```
bison -d calc.y
flex calc.l
gcc calc.tab.c lex.yy.c -o calc
./calc
```

Input & Output:

Test Case–1:

Input: 2 + 3 * 4

Output:

= 14

Test Case–2:

Input: (2 + 3) * 4

Output:

= 20

5. Result

The arithmetic expression parser was successfully implemented. The parser correctly evaluated expressions respecting standard operator precedence (* and / before + and −) and handled parentheses for grouping.

6. Conclusion

This experiment demonstrated how Flex and Bison cooperate to form the lexical analysis and syntax analysis phases of a compiler. The grammar-based approach of Bison ensures correct precedence and associativity in expression parsing.

Experiment No. 07: Flex & Bison — Variable Assignment Parser with Symbol Table

1. Objective

- To parse variable assignment statements using Flex and Bison.
- To implement a symbol table to store and retrieve variable values.
- To evaluate expressions involving previously assigned variables.

2. Software Requirement

- Flex 2.6.4 and Bison 3.x
- GCC C Compiler
- Operating System: Linux / Windows WSL

3. Theory

A symbol table is a data structure used by a compiler to store information about identifiers (variables). In this experiment, a simple array sym[26] maps each lowercase letter to its integer value. The parser recognizes assignment statements of the form: ID = expr; and stores the evaluated result in the symbol table for later use.

4. Program Implementation

Flex File (assign.l) — Lexical Analyzer:

Code:

```
%{
#include "assign.tab.h"
%}
%%
[a-z]   { yylval = yytext[0] - 'a'; return ID; }
[0-9]+  { yylval = atoi(yytext); return NUMBER; }
"="    return '=';
";"    return ';';
[+\-]   return yytext[0];
[ \t\r\n] ;
.       return yytext[0];
%%
int yywrap() { return 1; }
```

Bison File (assign.y) — Parser:

Code:

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
int sym[26];
%}
%token ID NUMBER
```

```
%%
program: program stmt | stmt ;
stmt:
   ID '=' expr ';'
   { sym[$1] = $3; printf("%c = %d\n", $1 + 'a', $3); }
   ;
expr:
   expr '+' expr  { $$ = $1 + $3; }
   | expr '-' expr  { $$ = $1 - $3; }
   | NUMBER
   | ID           { $$ = sym[$1]; }
   ;
%%
void yyerror(const char *s){ printf("Error: %s\n", s); }
int main() { printf("Enter assignments:\n"); yyparse(); return 0; }
```

Compile & Run Commands:

```
bison -d assign.y
flex assign.l
gcc assign.tab.c lex.yy.c -o assign
./assign
```

Input & Output:

Input:

```
a = 10;
b = a + 25;
a = a + b;
```

Expected Output:

Output:

```
a = 10
b = 35
a = 45
```

5. Result

The variable assignment parser was successfully implemented. The symbol table correctly stored and retrieved variable values, and expressions involving previously assigned variables were evaluated as expected.

6. Conclusion

This experiment demonstrated how a symbol table can be implemented alongside a parser to support variable assignments. The Bison grammar rules directly manipulate the symbol table through semantic actions, illustrating how meaning (semantics) is attached to syntax during compilation.

Experiment No. 08: Flex & Bison — Boolean Expression Parser

1. Objective

- To parse boolean expressions containing AND, OR, NOT, TRUE, FALSE.
- To implement operator precedence for boolean operators.
- To evaluate boolean expressions and print their result (0 or 1).

2. Software Requirement

- Flex 2.6.4 and Bison 3.x
- GCC C Compiler
- Operating System: Linux / Windows WSL

3. Theory

Boolean expressions use logical operators AND (&&), OR (||), and NOT (!). Operator precedence for boolean logic is: NOT has highest precedence, then AND, then OR. In Bison, this is declared explicitly using %right NOT, %left AND, %left OR directives. Bison's %left and %right declarations resolve shift/reduce conflicts automatically, ensuring correct evaluation order.

4. Program Implementation

Flex File (bool.l) — Lexical Analyzer:

Code:

```
%{
#include "def.tab.h"
%}
%%
"AND"   return AND;
"OR"    return OR;
"NOT"   return NOT;
"TRUE"  return TRUE;
"FALSE" return FALSE;
"("     return '(';
")"     return ')';
\n      return '\n';
[ \t]   ;  // ignore whitespace
.       { return yytext[0]; }
%%
int yywrap(){ return 1; }
```

Bison File (bool.y) — Parser:

Code:

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
%}
```

```
%token AND OR NOT TRUE FALSE
%left OR
%left AND
%right NOT
%%
input:
   input expr '\n' { printf("Result: %d\n", $2); }
   | expr '\n'     { printf("Result: %d\n", $1); }
   ;
expr:
   expr AND expr    { $$ = $1 && $3; }
   | expr OR expr   { $$ = $1 || $3; }
   | NOT expr       { $$ = !$2; }
   | TRUE           { $$ = 1; }
   | FALSE          { $$ = 0; }
   | '(' expr ')'  { $$ = $2; }
   ;
%%
void yyerror(const char *s){ printf("Syntax Error: %s\n", s); }
int main() { printf("Enter expression:\n"); yyparse(); return 0; }
```

Compile & Run Commands:

```
bison -d bool.y
flex bool.l
gcc bool.tab.c lex.yy.c -o bool
./bool
```

Input & Output:

Test Case–1:

Input: TRUE AND NOT FALSE

Output:

Result: 1

Test Case–2:

Input: (TRUE OR FALSE) AND FALSE

Output:

Result: 0

5. Result

The boolean expression parser was successfully implemented. Boolean expressions with AND, OR, and NOT operators were correctly evaluated, and parenthesized grouping was handled as expected.

6. Conclusion

This experiment demonstrated the use of Bison precedence declarations (%left, %right) to resolve ambiguities in boolean expression grammars. The combined Flex-Bison system successfully performed tokenization and parsing of boolean logic expressions.

Experiment No. 09: Flex & Bison — If-Else Statement Parser

1. Objective

- To parse simple if-else statements using Flex and Bison.
- To implement a symbol table for storing variable assignments.
- To handle the dangling-else ambiguity using %prec and %nonassoc directives.
- To evaluate conditional execution of assignment statements.

2. Software Requirement

- Flex 2.6.4 and Bison 3.x
- GCC C Compiler
- Operating System: Linux / Windows WSL

3. Theory

The if-else statement is a fundamental control flow construct in programming languages. When parsing nested if-else statements, a common ambiguity called the dangling-else problem arises: an else clause may be matched to either the nearest or the outermost if. Bison resolves this using %nonassoc IFX and %nonassoc ELSE, giving ELSE higher precedence than a standalone IF, so else binds to the nearest if statement.

The %union directive in Bison is used to define a union type for token values, supporting both integer (num) and character (id) token types in the same parser.

4. Program Implementation

Flex File (ifelse.l) — Lexical Analyzer:

Code:

```
%{
#include "ifelse.tab.h"
#include <stdlib.h>
%}
%%
"if"     return IF;
"else"   return ELSE;
[a-z]    { yylval.id = yytext[0]; return ID; }
[0-9]+   { yylval.num = atoi(yytext); return NUMBER; }
"="      return '=';
";"      return ';';
"("      return '(';
")"      return ')';
[ \t\n] ;
.        return yytext[0];
%%
int yywrap(){ return 1; }
```

Bison File (ifelse.y) — Parser:

Code:

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
int sym[26];
char last_id;
%}
%union { int num; char id; }
%token IF ELSE
%token <id> ID
%token <num> NUMBER
%type <num> expr stmt
%nonassoc IFX
%nonassoc ELSE
%%
program: program stmt | stmt ;
stmt:
   IF '(' expr ')' stmt %prec IFX
   { if($3){ printf("%c = %d\n", last_id, $5); } }
   | IF '(' expr ')' stmt ELSE stmt
   {
      if($3) printf("%c = %d\n", last_id, $5);
      else   printf("%c = %d\n", last_id, $7);
   }
   | ID '=' NUMBER ';'
   { last_id = $1; sym[$1-'a'] = $3; $$ = $3; }
   ;
expr: NUMBER { $$ = $1; } ;
%%
void yyerror(const char *s){ printf("Error: %s\n", s); }
int main(){ printf("Enter statements:\n"); yyparse(); return 0; }
```

Compile & Run Commands:

```
bison -d ifelse.y
flex ifelse.l
gcc ifelse.tab.c lex.yy.c -o ifelse
./ifelse
```

Input & Output:

Test Case–1:

Input: IF (1) a = 10; ELSE a = 20;

Output:

```
a = 10
```

Test Case–2:

Input: IF (0) a = 10; ELSE a = 20;

Output:

```
a = 20
```

5. Result

The if-else statement parser was successfully implemented. The dangling-else ambiguity was correctly resolved using %nonassoc directives, and the conditional execution of assignment statements produced correct outputs for both branches.

6. Conclusion

This experiment demonstrated how Flex and Bison can be used to parse control flow structures such as if-else statements. The use of %union, %nonassoc, and %prec directives in Bison is essential for handling ambiguous grammars and mixed token value types in real-world programming language parsers.

Experiment No. 10: Flex & Bison — List of Numbers Parser

1. Objective

- To parse a comma-separated list of numbers using Flex and Bison.
- To understand how recursive grammar rules are written in Bison.
- To print each parsed number individually.

2. Software Requirement

- Flex 2.6.4 and Bison 3.x
- GCC C Compiler
- Operating System: Linux / Windows WSL

3. Theory

A list grammar is a simple recursive Context-Free Grammar that recognizes sequences of items separated by delimiters. In this experiment, the grammar recognizes comma-separated integers. Bison left-recursive rules are preferred because they are more efficient for LALR(1) parsing — they do not cause stack overflow for long lists as right-recursive rules might.

Grammar for the list:

```
list : list ',' NUMBER   { printf("%d\n", $3); }
     | NUMBER            { printf("%d\n", $1); }
     ;
```

4. Program Implementation

Flex File (list.l) — Lexical Analyzer:

Code:

```
%{
#include "list.tab.h"
%}
%%
[0-9]+  { yylval = atoi(yytext); return NUMBER; }
","    return ',';
[ \t\r\n] ;
.       return yytext[0];
%%
int yywrap() { return 1; }
```

Bison File (list.y) — Parser:

Code:

```
%{
#include <stdio.h>
#include <stdlib.h>
int yylex();
void yyerror(const char *s);
%}
%token NUMBER
```

```
%%
list:
   list ',' NUMBER  { printf("%d\n", $3); }
   | NUMBER         { printf("%d\n", $1); }
   ;
%%
void yyerror(const char *s){ printf("Error: %s\n", s); }
int main(){ yyparse(); return 0; }
```

Compile & Run Commands:

```
bison -d list.y
flex list.l
gcc list.tab.c lex.yy.c -o list
./list
```

Input & Output:

Input: 1,2,3,4,5

Expected Output:

```
1
2
3
4
5
```

5. Comparison: Flex-Only vs Flex + Bison

Aspect	Flex Only	Flex + Bison
Scope	Lexical Analysis only	Lexical + Syntax Analysis
Token Classification	Yes — pattern matching	Yes — via Flex scanner
Expression Evaluation	Not possible	Yes — via semantic actions
Grammar Support	Regular Grammars only	Context-Free Grammars
Complexity Handling	Limited	Full language constructs
Use Case	Tokenizer, scanner	Full compiler front-end

6. Result

The list of numbers parser was successfully implemented. The comma-separated input was correctly parsed and each number was printed on a separate line, confirming the correct behaviour of the left-recursive Bison grammar rule.

7. Conclusion

This experiment demonstrated the use of recursive Bison grammar rules for parsing structured data such as comma-separated lists. Combined with the Flex scanner, the Flex-Bison toolchain provides a complete and efficient solution for implementing the front end of a compiler or interpreter.

…….End……